VoltEngine Billing API

A plain-language guide to allowlisting clients and sending billing requests.

What this API does

VoltEngine accepts account, meter, and tariff data, runs the billing calculation, and returns the charge breakdown, updated balances, and ledger lines. It provides daily billing, monthly invoice summaries, and wallet recharge endpoints.

How a request works

1. Your app sends JSON	2. Key and IP are checked	3. Billing is calculated	4. Result and ledger are returned

This service is stateless. Your system must save the returned wallet balance, arrear balance, and ledger entries. VoltEngine does not store account records.

Access: two allowlists are required

Each billing request must pass both checks. A correct API key alone is not enough, and an approved IP alone is not enough. If either allowlist is missing, billing endpoints return HTTP 503 and remain unavailable.

1. API key: the request sends an approved value in the **X-API-Key** header.

2. Client IP: the request must come from an approved IP address or CIDR range.

Configure comma-separated values in the server environment:

```
VOLTENGINE_API_KEYS="client-key-one,client-key-two"
VOLTENGINE_ALLOWED_IPS="203.0.113.10,198.51.100.0/24"
```

Replace the example addresses with real client IPs. Generate strong random API keys, share them only with approved clients, and never commit keys to source control. CIDR is a network range such as 198.51.100.0/24.

Run and explore the API

Install the API extra and start the server from the repository folder. For local development, use 127.0.0.1. For a deployed service, bind to 0.0.0.0 and set the port required by the host.

```
python -m pip install -e ".[api]"
python -m uvicorn http_api:app --host 127.0.0.1 --port 8000
```

Open /docs on the running server to see interactive request forms. The health check at /health is public. All /v1/billing/* routes require both allowlists.

Behind a reverse proxy? The IP check sees the address the ASGI server trusts. Configure forwarded-address handling only for your trusted proxy; do not trust client-supplied X-Forwarded-For headers.

Endpoints

Method and path	Purpose	Required input
GET /health	Confirm the service is running.	None. Public endpoint.
POST /v1/billing/daily	Calculate one daily prepaid bill.	Consumer, meter, tariff, period days, billing date.
POST /v1/billing/monthly	Summarize supplied ledger entries for a month.	Consumer ID, balances, month, ledger lines.
POST /v1/billing/recharge	Add funds to a wallet and optionally revise installment.	Consumer ID, current balances, amount.

Daily billing input

The consumer object carries the current wallet, arrear, sanctioned load, and optional installment. The meter object has daily units and maximum demand. The tariff object contains rate slabs and charge settings. The last slab must have **upto: null** so all usage can be priced.

Daily response

The response has a result object with totalDeduction, charge breakup, and updated state balances, plus a ledger array describing the entries posted by this calculation. Save these returned balances before the next billing request.

Daily billing: complete example

Send this JSON to **POST /v1/billing/daily**. Include the approved key as a header. Values below are illustrative; use the actual account and tariff data for your utility.

```
{
  "consumer": {
    "consumer_id": "C-1001",
    "wallet_balance": 500.0,
    "arrear_balance": 120.0,
    "load_kw": 3.0,
    "installment": {"total": 120.0, "daily": 2.0, "tenureDays": 60}
  },
  "meter": {"daily_units": 8.5, "max_demand_kw": 2.4},
  "tariff": {
    "slabs": [
      {"upto": 50, "rate": 3.1},
      {"upto": 100, "rate": 3.6},
      {"upto": null, "rate": 4.1}
    ],
    "fixed_charge": 120,
    "duty_rate": 0.05,
    "dps_monthly_rate": 0.015,
    "demand_rate": 250,
    "excess_demand_multiplier": 1.5
  },
  "period_days": 30,
  "billing_date": "2026-11-06"
}
```

Example response shape

```
{
  "result": {
    "totalDeduction": 33.73,
    "breakup": {"energy": 26.35, "fixed": 4.0, "duty": 1.32,
                "dps": 0.06, "installment": 2.0},
    "state": {"walletBalance": 466.27, "arrearBalance": 118.06}
  },
  "ledger": [
    {"date": "2026-11-06", "type": "ENERGY", "amount": 26.35, "balance": 466.27}
  ]
}
```

These sample amounts correspond to the example input on the previous page. Your result changes with the consumer, tariff, and meter values you send.

Monthly summary and recharge

Monthly invoice request

Send **POST /v1/billing/monthly** with the month in YYYY-MM format and the ledger lines to summarize. Accepted line types include ENERGY, FIXED, DUTY, DPS, INSTALLMENT_RECOVERY, and EXCESS_DEMAND_PENALTY. The service normalizes letter case.

```
{
  "consumer_id": "C-1001",
  "wallet_balance": 466.27,
  "arrear_balance": 118.06,
  "month": "2026-11",
  "ledger_entries": [
    {"type": "ENERGY", "amount": 26.35},
    {"type": "FIXED", "amount": 4.0},
    {"type": "DUTY", "amount": 0.79}
  ]
}
```

Recharge request

Send **POST /v1/billing/recharge** with current account state and a positive amount. The response contains the new wallet balance and installment. Save the returned state in your account system.

```
{
  "consumer_id": "C-1001",
  "wallet_balance": 466.27,
  "arrear_balance": 118.06,
  "load_kw": 3.0,
  "installment": {"total": 120.0, "daily": 2.0, "tenureDays": 60},
  "amount": 500.0,
  "revise_installment": true
}
```

Common errors

HTTP status	Meaning and action
401	Missing or incorrect API key. Send a valid X-API-Key header.
403	Client address is not allowlisted. Add the caller IP or its CIDR range to VOLTENGINE_ALLOWED_IPS.
422	Input is invalid. Check required fields, non-negative values, dates, and final open-ended tariff slab.
503	One or both allowlists are missing or malformed. Configure both server environment variables.

Important: VoltEngine does not save balances or prevent your application from recording the same returned result twice. Persist every successful response once, maintain an audit trail, and send the latest balances on the next request.

For developers: GET /docs provides interactive OpenAPI documentation while the service is running. Keep production keys private and manage the allowlists through deployment settings.